Olist E-Commerce Data Chatbot

Complete Project Documentation
& Interview Preparation Guide

Python | PostgreSQL | Pandas | LangChain | Groq | FastAPI | Streamlit | BeautifulSoup

GitHub: github.com/atishay01/Ai_agent

Author: Atishay Sharma

Part 1 — Project Documentation

1. Overview

An AI-powered data-query chatbot over the public Brazilian E-Commerce (Olist) dataset from Kaggle. Users type business questions in plain English; a LangChain SQL agent writes SQL, runs it against PostgreSQL, optionally calls web APIs for currency or geographic information, and returns a readable answer.

2. Architecture

```
9 CSVs ──► ETL (Pandas) ──► PostgreSQL (8 tables, ~570K rows)
                                    │
                                    ▼
                          LangChain SQL Agent (Groq)
                          ├── SQL Toolkit
                          ├── get_usd_brl_rate    (requests + JSON API)
                          ├── lookup_brazilian_state  (BeautifulSoup + Wikipedia)
                          └── calculate            (exact arithmetic)
                                    │
                                    ▼
                           FastAPI  ──►  Streamlit UI
                            :8000          :8501
```

3. Tech Stack

Layer	Choice	Why
Language	Python 3.11	Rich ecosystem for data + AI
Warehouse	PostgreSQL 16	Relational data, FK integrity, industry standard
ETL	Pandas, SQLAlchemy	Fast prototyping for CSV-to-DB
LLM	Groq (openai/gpt-oss-20b)	Free tier, low latency, OpenAI-compatible API
Agent	LangChain create_sql_agent	Battle-tested text-to-SQL loop
Backend	FastAPI	Fast, typed, simple to test

Frontend	Streamlit (themed, wide layout)	Zero-boilerplate chat UI
Web data	Requests, BeautifulSoup	Matches the requirement

4. Data Model (3NF)

Dimensions: customers, sellers, products, geolocation

Facts: orders, order_items, order_payments, order_reviews

Foreign keys and indexes on every join column. Full DDL in `src/schema.sql`.

Row counts after ETL

Table	Rows
customers	99,441
sellers	3,095
products	32,951
geolocation	19,015 (deduped from ~1M raw)
orders	99,441
order_items	112,650
order_payments	103,886
order_reviews	99,224
Total	**569,703**

5. ETL Pipeline Stages

1. **Extract** — 9 raw CSVs read with Pandas.
2. **Profile** — shape, null counts, duplicate counts printed.
3. **Clean** — timestamp parsing, whitespace trim, state code normalisation, zip padding, null-dimension fill with category median.
4. **Transform** — derived columns (delivery_days, is_late), English category name join, geolocation aggregation (mean lat/lng per zip).
5. **Load** — SQLAlchemy to_sql in 5,000-row batches, in FK dependency order.

6. **Validate** — re-query Postgres row counts, compare.

6. Setup Instructions

```
# 1. Clone the repo
git clone https://github.com/atishay01/Ai_agent.git
cd Ai_agent

# 2. Create venv and install deps
py -3.11 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# 3. Download the 9 Olist CSVs from Kaggle into data/raw/

# 4. Create database
psql -U postgres -c "CREATE DATABASE olist_db;"

# 5. Copy .env.example to .env and fill in your values

# 6. Run ETL (~90 seconds, loads 570K rows)
python src/etl.py

# 7. Start servers (two terminals)
python src/api.py              # FastAPI on :8000
streamlit run src/app.py       # Streamlit on :8501

# 8. Open http://localhost:8501
```

7. Design Notes

Prompt Design

The agent receives a concise system prompt listing the schema plus a few column-usage rules (for example, `category_name_en` for category filters, `is_late = TRUE` for late deliveries). This trims the LLM's search space and produces more reliable SQL than a bare schema dump.

LLM Math Drift

Multiplying a 7-digit BRL total by an exchange rate causes LLM arithmetic to drift by 0.1–1%. The `calculate` tool evaluates expressions through Python's AST module, making currency conversions

exact. During testing, a USD conversion was off by $4,734 until I added this tool — after adding it, the answer matches hand calculation to the cent.

Agent Iteration Cap

Questions like "top 5 states with capitals" need at least 7 tool calls (one SQL + five Wikipedia lookups + a final answer). My initial `max_iterations=10` was too tight — the agent bailed mid-answer with "Agent stopped due to max iterations". I raised the cap to 20 and added prompt rules ("don't re-inspect the schema unnecessarily") so complex questions finish inside the budget. This is a classic agent-tuning trade-off: lower cap fails fast on runaway loops, higher cap handles multi-step questions.

Web Data Integration

Two external sources: the Frankfurter API (JSON over HTTPS, using `requests`) for live BRL/USD rates, and the Wikipedia "States of Brazil" page (parsed with `BeautifulSoup`) for state names and capitals. The database has no `capital` column — any answer containing a capital name must come from the scraper, proving the tool ran.

UI & Theme

The Streamlit app uses a wide layout, a custom theme defined in `.streamlit/config.toml` (dark-blue primary colour, soft off-white sidebar), sample questions grouped into "Database queries" and "With live web data", a one-click "Clear chat" button, and a collapsible "Tables loaded" panel to keep the sidebar uncluttered.

8. Limitations (Honest)

- Only tested manually; no automated test suite.
- Free Groq tier has a daily token cap; heavy use falls back to a friendly error in FastAPI.
- Wikipedia scraping depends on page structure; layout changes would need a code update.
- Runs locally only. Full deployment would need Postgres on Supabase, FastAPI on Render, Streamlit on Streamlit Cloud.

9. Project Layout

```
.
├── .streamlit/config.toml  # Streamlit theme (wide layout, brand colour)
├── .vscode/                # editor settings and F5 debug configs
├── data/raw/               # 9 Olist CSVs (gitignored)
├── queries/                # sample SQL for pgAdmin demos
├── src/
│   ├── schema.sql          # DDL
│   ├── db.py               # Postgres connection helper
│   ├── etl.py              # Pandas ETL pipeline
│   ├── web_tools.py        # currency, Wikipedia, calculator
│   ├── agent.py            # LangChain SQL agent
│   ├── api.py              # FastAPI backend
│   └── app.py              # Streamlit chat UI
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

Part 2 — Interview Preparation Guide

A. The 30-Second Elevator Pitch (Memorise This)

> *I built a data-query chatbot over the Olist Brazilian e-commerce dataset. A user types a natural-language question in a Streamlit chat UI. That goes to a FastAPI backend, which passes it to a LangChain SQL agent running on Groq. The agent writes SQL, runs it against a PostgreSQL warehouse of about 570,000 rows across 8 tables, and can also call web tools — a currency API and a Wikipedia scraper — when the question needs data outside the database. The answer comes back in plain English.*

B. Architecture & Tech-Choice Questions

Q: Why PostgreSQL and not MongoDB or SQLite?

> *The Olist data is fundamentally relational — customers, orders, items, products all connect via foreign keys. PostgreSQL gives me proper joins, constraints, indexes, and it's the industry default for transactional analytics. SQLite would have been simpler but doesn't handle concurrent connections well if I later expose this to users.*

Q: Why Groq and not OpenAI or Claude?

> *Three reasons: free tier was enough for development, latency is very low — they use specialised inference hardware — and it exposes an OpenAI-compatible API so switching providers is a one-line change in* `.env`*.*

Q: Why FastAPI if Streamlit can talk to the DB directly?

> *Separation of concerns. Streamlit is just the UI; FastAPI is the business logic. If tomorrow I wanted to add a WhatsApp bot or a mobile app, they'd hit the same* `/query` *endpoint — zero rewrite. Also, FastAPI makes error handling, logging and authentication much cleaner than pushing it into the UI.*

Q: Why LangChain instead of writing the SQL prompt yourself?

> *LangChain's `create_sql_agent` handles the multi-step reasoning loop — inspecting schema, choosing tables, writing SQL, handling errors, retrying. Reimplementing that correctly would have been days of work. I used the framework but customised the system prompt to encode domain-specific rules about my schema.*

C. ETL Pipeline Questions

Q: Walk me through your ETL pipeline.

> *Six stages in `etl.py`. Extract reads the nine raw CSVs with Pandas. Profile prints shape, null counts, duplicate counts so I can see what I'm dealing with. Clean handles each type of issue — timestamp parsing, trimming whitespace, normalising state codes to uppercase, filling null product dimensions with category medians. Transform adds derived columns like `delivery_days` and `is_late`. Load uses SQLAlchemy's `to_sql` with multi-row inserts in batches of 5000. Validate re-queries Postgres to confirm row counts match what we sent.*

Q: How did you handle missing data?

> *Two strategies. For product dimensions — weight, length, height, width — about 2,400 rows had nulls. I grouped by category and filled each null with that category's median. For rows in a category with no non-null data at all, I fell back to the global median. For order timestamps — some deliveries were never completed — I kept them as NULL, which propagates correctly through my derived `is_late` calculation.*

Q: You have ~1 million geolocation rows but only 19,000 made it in. Why?

> *The raw geolocation CSV has multiple lat/long entries per zip code — different street addresses sharing a ZIP prefix. For analytical use, I only need one representative coordinate per zip. So I grouped by zip code and aggregated — mean latitude and longitude, modal city and state — collapsing 1M rows into 19K.*

Q: Why 3NF and not a star schema?

> *Olist's size, about half a million rows, doesn't justify denormalisation. Third normal form keeps the data clean and avoids update anomalies. If this were a 50-million-row warehouse with heavy read patterns, I'd consider denormalising hot paths.*

D. LangChain / Agent Questions

Q: How does the agent decide which tables to query?

> *LangChain's SQL toolkit gives the LLM three sub-tools automatically — `list_tables`, `schema_of_table`, and `execute_sql`. It follows a ReAct-style loop: look at the user's question, list available tables, fetch the schemas it thinks are relevant, draft SQL, execute it, then summarise. I also give it a concise system prompt listing my 8 tables and specific rules like "use `category_name_en` for category filters" — this eliminates ambiguity and cuts wrong queries significantly.*

Q: How does the agent prevent SQL injection or destructive queries?

> *Two layers. First, my system prompt explicitly forbids UPDATE, DELETE, INSERT, DROP, and ALTER. Second, the Postgres connection is scoped — in production I'd use a read-only role for the agent's connection. For this demo I used the default postgres role but the architectural hook is in place.*

Q: What if the LLM hallucinates a wrong answer?

> *That's the real risk with text-to-SQL. I mitigated it with three things. First, grounding the prompt in the actual schema with two sample rows per table — so the LLM sees real column names and value formats. Second, the `calculate` tool — LLMs drift on large-number arithmetic, so I force them to delegate multiplication to Python. Third, I log every SQL query the agent runs, so I can audit what it actually did after the fact.*

E. Web Scraping Questions

Q: Why scrape Wikipedia for state info? Why not hardcode it?

> *Two reasons. One, it was the task's requirement — the proposal said BeautifulSoup. Two, Wikipedia is the canonical source. If Brazil ever reorganises its states, Wikipedia updates first. A hardcoded dict would silently go stale.*

Q: What happens if the scraper fails?

> *It returns a graceful error message — "No Brazilian state found" — rather than crashing the agent. The agent sees the error and can still produce a partial answer based on the SQL result alone.*

Q: What's the difference between your currency tool and your scraper tool?

> *The currency tool calls a JSON API — Frankfurter — using `requests`. Clean, structured, fast. The scraper uses `requests + BeautifulSoup` to parse an HTML page. Both demonstrate pulling external data, but the API call is deterministic while the scraper depends on page structure — which is why scrapers are more fragile in production.*

F. Behavioural / Honest Questions

Q: Did anyone help you build this?

> *I used documentation heavily — LangChain, FastAPI, Streamlit — and AI coding assistants like ChatGPT and Claude to speed up boilerplate and debug errors. Every architecture decision is mine, and I can walk through any line of code. Using AI tools is standard for developers today; hiding it would be dishonest.*

Q: How long did this take you?

> *About two evenings of focused work. Setup and database creation on the first evening, ETL pipeline and agent on the second, UI and web tools on the morning of submission.*

Q: Did anything break during testing, and how did you fix it?

> *Two things, actually. First, my USD conversions were drifting by about 0.4% because the LLM was doing the BRL × rate multiplication itself — I fixed it by adding a `calculate` tool that evaluates the expression through Python's AST module, so the math is exact. Second, when I asked "top 5 states with capitals," the agent stopped mid-answer with "max iterations reached" — because the question needs 6+ tool calls and my limit was 10. I raised the limit to 20 and added a prompt rule telling the agent not to re-inspect the schema unnecessarily. Both are classic agent-system issues and taught me a lot about where LLMs need deterministic helpers.*

Q: What was the hardest part?

> *Getting the agent to produce correct SQL on a non-trivial schema without hallucinating. The solution was prompt engineering — listing the 8 tables explicitly, stating column-usage rules, and giving the agent two sample rows per table. Accuracy improved noticeably after that.*

Q: What would you do differently with more time?

> *Three things. First, automated tests — both for the ETL and for the agent's SQL generation. Second, deploy it properly — Postgres on Supabase, FastAPI on Render, Streamlit on Streamlit Cloud. Third, add observability — currently I just log to stdout, but I'd want structured logs so I could analyse which question types the agent struggles with.*

Q: Why didn't you use all the CSVs at first?

> *I originally scoped to four core tables because time was tight — customers, products, orders, order_items. Once the pipeline was working, I extended to all eight because the marginal cost of adding a table is small when the framework is in place. That's an incremental-delivery mindset — get the end-to-end working first, then expand.*

G. Demo Script — what to show, in what order

1. **Open GitHub repo** — "Here's the code, public repo at github.com/atishay01/Ai_agent. README has the architecture."
2. **Open VS Code** at `src/schema.sql` — "3NF data model, 8 tables with FKs."
3. **Show** `src/etl.py` — scroll through, highlight `build_products` — "6-stage pipeline with Pandas."
4. **Open pgAdmin** — expand `olist_db`, show tables, right-click `orders` → View Data.
5. **Run a query** from `queries/02_business_questions.sql` — "Same SQL the agent generates."
6. **Show** `src/agent.py` — "75 lines. LangChain's SQL agent plus three custom tools."
7. **Show** `src/web_tools.py` — "`requests` for the rate, `BeautifulSoup` for Wikipedia, AST for exact math."
8. **Switch to the Streamlit app** (`localhost:8501`) — ask 3 questions:
    - "Top 5 product categories by revenue?" — pure SQL
    - "What is the capital of state RJ?" — scraper
    - "Total revenue in USD?" — SQL + currency + calculator

9. **Show FastAPI terminal logs** — "Every tool call is logged — `calculate` and `scraped 27 Brazilian states from Wikipedia` entries prove the tools actually ran."

H. Final Checklist for Tomorrow

- **Sleep 6+ hours.** Tired demos fail.
- **Morning warm-up:** start both servers, test 3 questions, confirm all is well.
- **Speak slowly.** You know this code. Don't rush.
- **If you don't know something, say so.** "I don't know — I'd look it up" beats bluffing every time.
- **Have the GitHub link ready** — interviewer may ask for it: `github.com/atishay01/Ai_agent`

End of document. Good luck.